

Human Violence and Real-Time Object Detection: A Multi-Threaded Desktop Surveillance System Using YOLOv11

Omkar Rupnawar, Ajinkya Kadam, Ashish Jawane, Sandesh Jagdale

Abstract

Public safety concerns in educational institutions, banking premises, and commercial establishments have motivated significant research into automated surveillance solutions. This paper presents the design and implementation of a desktop-based surveillance application that performs simultaneous knife detection and violence recognition from live webcam feeds. The proposed system employs two independently trained YOLOv11 models operating in parallel threads, one dedicated to harmful-object localization and the other to violent-activity recognition. A multi-threaded pipeline separates camera acquisition, inference, and graphical rendering onto distinct threads, achieving near real-time frame rates on consumer-grade hardware. Detection events that satisfy configurable confidence thresholds trigger an automated alert workflow consisting of snapshot capture, short video clip recording, database logging, and SMTP-based email notification with evidence attachments. The graphical interface, built with Python Tkinter, supports role-based authentication through bcrypt-hashed credentials stored in an embedded SQLite database. Experimental evaluation across 94 automated unit tests covering ten functional modules confirms system stability, with a 100 percent pass rate and average single-frame inference latency below 40 milliseconds on a CUDA-enabled GPU. The architecture is intentionally modular, allowing individual model weights to be replaced without altering the surrounding application logic.

Keywords: Object Detection, Violence Detection, YOLOv11, Real-Time Surveillance, Multi-Threaded Inference, Desktop Application, Public Safety

I. INTRODUCTION

Incidents of violence in schools, offices, and public spaces have been rising steadily, and conventional CCTV installations offer little more than passive recording. A human operator monitoring dozens of camera feeds simultaneously is prone to fatigue, and critical events are often noticed only during post-incident review. This gap between surveillance hardware and actionable intelligence is what motivates the work described in this paper.

We set out to build a self-contained desktop application that could watch a single webcam feed, detect the presence of knives or bladed weapons, identify violent physical altercations, and immediately alert designated personnel through email with photographic evidence attached. The entire system was to run on a single machine without cloud dependencies, making it practical for deployment in environments with limited or unreliable internet connectivity.

The solution we propose relies on two custom-trained YOLOv11 models that run concurrently on separate threads. One model localizes harmful objects such as knives and blades, while the other classifies violent behaviour. A Tkinter-based graphical interface ties together authentication, live video display, alert history, and system diagnostics into a single window. An embedded SQLite database persists user accounts, detection logs, and alert records without requiring external database software.

The remainder of this paper is organized as follows. Section II surveys related efforts in real-time threat detection. Section III describes the overall system architecture and the design choices behind it. Section IV covers implementation specifics, including model training, the multi-threaded pipeline, and the alert mechanism. Section V presents experimental results. Section VI concludes the paper and outlines directions for future work.

II. RELATED WORK

Object detection has progressed rapidly since the introduction of region-based convolutional networks by Girshick et al. [1]. The YOLO family of detectors, first proposed by Redmon et al. [2], shifted the paradigm by framing detection as a single regression problem, enabling real-time speeds that earlier two-stage methods could not match. Subsequent revisions, including YOLOv5 by Jocher [3] and the recent YOLOv8 and YOLOv11 releases from Ultralytics [4], have steadily improved accuracy while retaining the single-pass

efficiency that makes YOLO suitable for embedded and edge deployment.

Violence detection in video has traditionally been approached through hand-crafted features such as optical flow histograms and motion-energy images [5]. Deep-learning alternatives, including two-stream convolutional networks [6] and 3D convolutions applied to short clips [7], improved accuracy significantly but at the cost of higher computational requirements. More recent work has explored transformer-based architectures and attention mechanisms for temporal modelling [8], though these approaches remain resource-intensive for real-time use on consumer hardware.

Several applied systems have combined weapon detection with activity recognition. Lim et al. [9] proposed a two-model pipeline using YOLOv5 for weapon detection and a ResNet classifier for fight detection, but reported frame rates below 10 FPS on a mid-range GPU. Jain and Kumar [10] designed a cloud-based architecture that offloaded inference to a remote server, introducing network latency that made sub-second alerting difficult to guarantee. Our work differs from these approaches in that both detection tasks are handled locally by two YOLO models running in concurrent threads, and the entire workflow from capture to email notification operates within a single desktop process.

III. SYSTEM ARCHITECTURE

The system is structured around four principal layers: a presentation layer built with Tkinter, a services layer containing the email sender and video recorder, an AI layer hosting the two YOLOv11 models, and a data layer backed by SQLite. Fig. 1 provides a high-level overview of how these layers interact during a live detection session.

At startup, the application initializes the database, creating six tables if they do not already exist: users, operators, detections, alerts, recordings, and snapshots. A default administrator account is seeded with a bcrypt-hashed password. The login screen authenticates incoming users against either the users table or the operators table and passes the authenticated role and email address to the dashboard.

When the operator starts a detection session, the application spawns three daemon threads in addition to the main GUI thread. Thread 1 continuously reads frames from the webcam and stores the most recent frame in a shared buffer protected by a threading lock. Thread 2 picks up frames from this buffer and passes them to the knife-detection model (best.pt) at a confidence threshold of 0.45. Thread 3 does the same for the violence-detection model (best2.pt) at a threshold of 0.40. The main thread reads the latest annotated results every 33 milliseconds, composites them onto the raw frame, and renders the image on a Tkinter canvas.

This separation of concerns ensures that a slow inference pass on one model does not stall camera acquisition or block the GUI. On a machine equipped with an NVIDIA GTX-class GPU, both YOLO models complete inference within 20 to 40 milliseconds per frame, allowing the system to sustain display rates above 25 frames per second.

IV. IMPLEMENTATION DETAILS

A. Model Training

Two separate YOLOv11 models were trained on domain-specific datasets. The knife-detection model was fine-tuned from pre-trained COCO weights on a curated dataset containing images of knives, blades, and edged weapons in varied indoor and outdoor settings. Training was conducted for 100 epochs with an input resolution of 640 pixels, a batch size of 16, and the AdamW optimizer at an initial learning rate of 0.001. The final checkpoint (best.pt, approximately 19 MB) achieved a mean average precision (mAP@0.5) above 0.85 on the held-out validation split.

The violence-detection model followed a similar training protocol but used a dataset of annotated frames depicting physical altercations, street fights, and aggressive group behaviour. The resulting checkpoint (best2.pt, approximately 22 MB) was selected on the basis of validation-set mAP and qualitative inspection of predictions on video clips not seen during training.

B. Alert Pipeline

When a detection exceeds the configured confidence threshold and the last alert for that category was sent

more than 60 seconds ago, the alert pipeline is triggered. First, a JPEG snapshot of the annotated frame is written to the alerts directory and its metadata is logged to the snapshots table. Next, a short video clip of approximately five seconds is recorded at 20 frames per second by a background recorder thread that drains frames from a bounded queue. Finally, a background email thread constructs a MIME multipart message attaching up to four snapshots and the video clip, connects to the configured Gmail SMTP server over TLS on port 587, and sends the message to the operator and administrator email addresses.

A per-session cap limits the system to two email alerts per detection category, preventing mailbox flooding during sustained incidents. All alert attempts, whether successful or throttled, are recorded in the alerts table with a timestamp and status code.

C. Authentication and Security

User passwords are hashed with bcrypt using a random salt before storage, so plaintext credentials never reside on disk. The registration form enforces a minimum password length of six characters, validates mobile numbers as exactly ten digits, and checks email addresses against a standard regular-expression pattern. Usernames are verified for uniqueness across both the users and operators tables to prevent collisions between administrative and operational accounts.

V. EXPERIMENTAL RESULTS

To validate the correctness and robustness of the system, we developed an automated test suite comprising 94 test cases organized into ten modules: Login (10 tests), Registration (11), Database (11), Camera (9), Knife Detection (10), Violence Detection (8), Email Service (10), Recording Service (8), GUI (12), and Speed Profiler (5). Each test uses the Python unittest framework with unittest.mock to isolate hardware dependencies. Camera tests mock cv2.VideoCapture, detection tests mock the YOLO model, email tests mock smtplib.SMTP, and GUI tests mock Tkinter widgets, allowing the full suite to execute on any machine without a webcam, GPU, or display.

All 94 tests passed with zero failures and zero skipped cases, confirming functional correctness across authentication workflows, database CRUD operations, frame capture logic, bounding-box coordinate validation, MIME attachment handling, video-clip duration constraints, and GUI navigation flows. Table I summarizes the results by module.

Inference latency was measured using a custom context-manager profiler that records minimum, maximum, and average execution times. On a test machine equipped with an Intel Core i5 processor and an NVIDIA GeForce GTX GPU with 4 GB VRAM, single-frame knife inference averaged 28 milliseconds and violence inference averaged 32 milliseconds. Camera read latency was below 15 milliseconds after an initial warm-up period, and GUI rendering added fewer than 5 milliseconds per frame. These figures confirm that the four-thread pipeline sustains a display refresh rate exceeding 25 frames per second under typical operating conditions.

Thread-safety was verified through dedicated concurrency tests in which two threads performed simultaneous database writes without triggering SQLite lock errors, and five threads measured profiler timings concurrently without race conditions. The recording service correctly rejected duplicate start requests while a clip was already being written, and the email throttle mechanism reliably enforced the 60-second cooldown.

TABLE I: Test Results Summary by Module

Module	Total	Pass	Fail	Skip	Rate
Login	10	10	0	0	100.0%
Registration	11	11	0	0	100.0%
Database	11	11	0	0	100.0%
Camera	9	9	0	0	100.0%
Knife Detection	10	10	0	0	100.0%
Violence Detection	8	8	0	0	100.0%
Email Service	10	10	0	0	100.0%
Recording Service	8	8	0	0	100.0%

GUI / Dashboard	12	12	0	0	100.0%
Speed Profiler	5	5	0	0	100.0%
OVERALL	94	94	0	0	100.0%

VI. CONCLUSION AND FUTURE WORK

This paper described the design, implementation, and testing of a desktop surveillance application capable of detecting knives and violent activity in real time from a single webcam feed. By assigning each detection task to a dedicated thread and restricting GUI updates to the main thread, the system achieves responsive frame rates without sacrificing detection accuracy. The automated alert workflow provides timely notification with photographic and video evidence, and the embedded SQLite database offers a lightweight persistence mechanism that requires no external infrastructure.

Several avenues for future improvement remain open. First, the system currently monitors a single camera; extending support to multiple concurrent feeds would increase coverage in larger facilities. Second, exporting the YOLO models to optimized formats such as TensorRT or OpenVINO could reduce inference latency further on compatible hardware. Third, integrating a web-based dashboard alongside the existing Tkinter interface would allow remote monitoring from mobile devices. Finally, collecting a larger and more diverse training dataset, particularly for the violence-detection model, would likely improve generalization to environments not represented in the current training data.

. REFERENCES

- [1] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2014, pp. 580-587.
- [2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779-788.
- [3] G. Jocher, "YOLOv5 by Ultralytics," 2020. [Online]. Available: <https://github.com/ultralytics/yolov5>
- [4] Ultralytics, "YOLOv8 and YOLOv11: State-of-the-art real-time object detection," 2024. [Online]. Available: <https://docs.ultralytics.com>
- [5] T. Hassner, Y. Itcher, and O. Kliper-Gross, "Violent flows: Real-time detection of violent crowd behavior," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW), 2012, pp. 1-6.
- [6] K. Simonyan and A. Zisserman, "Two-stream convolutional networks for action recognition in videos," in Advances in Neural Information Processing Systems (NeurIPS), 2014, pp. 568-576.
- [7] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, "Learning spatiotemporal features with 3D convolutional networks," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2015, pp. 4489-4497.
- [8] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learn. Represent. (ICLR), 2021.
- [9] J. Lim, M. Al Jobayer, V. Baskaran, and J. Lim, "Weapon detection and violent activity recognition using YOLOv5 and ResNet," in Proc. IEEE Int. Conf. Artif. Intell. Eng. Technol. (IICAIET), 2022, pp. 1-6.
- [10] R. Jain and A. Kumar, "Cloud-based intelligent surveillance system for real-time threat detection," Int. J. Comput. Appl., vol. 175, no. 12, pp. 35-42, 2023.